

The evolution of Android applications security

Abdenmour JEBBAR
IPSS TEAM,
Computer Science Department,
Faculty of Sciences, Mohammed V University in Rabat,
Rabat, Morocco
jebbar.abdenmour@gmail.com

Ahmed EL-YAHYAOU
IPSS TEAM,
Computer Science Department,
Faculty of Sciences, Mohammed V University in Rabat,
Rabat, Morocco
a.elyahyaoui@um5r.ac.ma

Abstract— Since its beta release in 2007, android had been growing both amongst users and developers. Its ease of use, customizability, programmability, and open source nature caught not just the attention of both tech enthusiasts and regular users, but also manufacturers who started using it in their products either in its original format like the case of Samsung, or for custom operating systems based on it such as ColorOS for Oppo or HarmonyOS for Huawei making it the number one used OS in the world. But this also created more space for security issues to occur and created a great environment for hackers. In this paper, we explore the evolution of security measures introduced in Android since its first release, both for the Software development kit (SDK) and operating system across various versions. Starting with an overview of Android Architecture, we will then explore a set of methods used to classify vulnerabilities with statistics from the National Vulnerability Database (NVD). We will then study the evolution of security measure introduced in each step of the growth of Android, from early static permission models to more sophisticated features such as post-quantum encryption, SELinux and others, which will bring us to also discuss the challenges that remain. As many android development frameworks emerge, we will do a comparative analysis of the security of the modern Android development frameworks notably Flutter and React native face to face with the native Android, putting their strengths and weaknesses in terms of built-in security features under the spotlight. During this comparison, we focus on storage, encryption, obfuscation, and compromised device detection. To sum up, we will present a conclusion summarizing the analysis with a perspective on future work and how it will help improve the current situation.

Keywords— *Android, Security, Flutter, React Native, vulnerability.*

I. INTRODUCTION

In this digital world, mobile applications have been a pillar of our daily lives, serving personal and professional needs. From transport apps to healthcare solutions play a vital role in modern society with an average mobile screen time of 3 hours and 15 minutes. Among the operating systems running on the 7 billion active devices, Android (and other operating systems build over it) is the most widespread, taking the biggest share in this market, with more than 72% of devices in 2024 [1]. Over the years, it evolved, and it became a complex and diversified ecosystem supported by different phone brands, smartwatches, and TVs. However, this dominance makes it a target for cyberattacks, making each flaw high-risk due to the wide populations reached in each attack. The attackers exploit a combination of OS vulnerabilities, insecure development practices, and hardware problems [2] to compromise the security of the apps and user data.

Improving Android applications' security is no longer merely a technical requirement but a societal necessity. As smart gadgets have become a vault filled with personal information, from financial data to medical records, private

communication, photos. With the adoption of policies such as Bring your own device (BYOD), the lack of security can even expose organizations data or expose them to breaches due to compromised devices.

Beyond the professional realm, the need of robust security measures is even more critical with the exposition of vulnerable demographics to digital devices in this era, significantly toddlers and elders who may lack the technical proficiency to recognize and mitigate threats. For instance, children interacting with gaming apps or educational ones could put them at risk of sharing confidential information or downloading malware that looks like a game, or elders falling victim to phishing attacks, social engineering, etc.

Despite the considerable progress made by Google, to improve both the system's security and its applications, the development process still faces multiple challenges. The modern frameworks, from their side, offer a level of flexibility, abstraction, and ease of use, but introduce limitations to interfacing with native APIs and built-in security supports, and bring with them new attack surfaces that are caused by this lack or inherited from the base language.

The human element also remains an important part of these challenges. The lack of knowledge of the security standards and practices ad mentioned in guides like OWASP MASVS [3] among developers of the best security practices and wrongful implementations, and the unawareness of the users with the risks they face while using their devices openly aggravates the risks of potential harmful attacks.

II. ANDROID'S ARCHITECTURE

A. Architecture layers

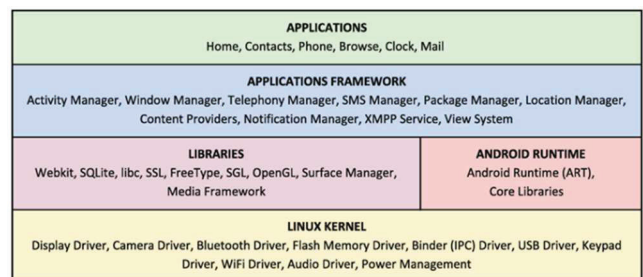


Fig. 1. Simplified Android system architecture.

As visible in Fig. 1, Android OS is composed of 4 layers:

- Kernel: a modified version of the Linux 2.6 Kernel [4]. This layer contains drivers, power management etc. In short, The Kernel layer does the basic tasks ensuring the system's operation.
- Middle Tier [5]: composed of Android Runtime (ART) and Libraries. Its main job is to provide

services like storage etc. through the framework's API.

- **Applications Framework:** it is a set of services, managers allowing developers to interact with the lower levels when building an application.
- **Applications:** made of the union of built-in and third-party applications. This layer is the front line that is handled by all smartphone users.

In this architecture, each layer is responsible for collecting its inputs from the layer below and providing an output to the one above. This puts the Android developer's mission right on the edge of the Applications Framework layer, giving him the privilege to focus on making use of this API to produce useful applications and securing them while Google and the contributors on Android OS focus on improving the rest of the layers and improving their security to provide a secure base to build upon.

III. METHODS OF CLASSIFYING VULNERABILITIES

Each day, lots of new vulnerabilities are discovered across multiple platforms. To study them, many indicators have been put in action for multiple purposes such as classification or prioritization.

A. Vulnerability types

TABLE I. NUMBER OF VULNERABILITIES DISCOVERED IN ANDROID PER CATEGORY IN THE LAST DECADE [6]

	Code execution	Bypass	Privilege escalation	Denial of Service	Information leak
2015	70	3	3	56	17
2016	72	33	59	106	87
2017	201	29	235	87	116
2018	58	1	63	34	106
2019	92	2	3	37	216
2020	40	14	17	49	273
2021	28	17	16	35	194
2022	17	54	69	107	290
2023	24	5	10	274	497
2024	16	0	6	64	133

Vulnerabilities can be unique, with tailored attack vectors and impact different blocks of the system, but they might keep some similar characteristics and traits, and they represent the purpose of the software vulnerability. In 'Table I' we have the number of vulnerabilities detected in Android by year for the following vulnerability types:

- **Code Execution:** this allows the attacker to execute arbitrary code on the victim's device [7].
- **Bypass:** allows the attacker to work around security controls etc. to perform tasks that he is not authorized to do [7].
- **Privilege escalation:** it allows an entity of a low role to gain a role with more privileges which can give it, for example, access to sensitive data or the rights to

execute tasks that can be critical, for example getting root privileges [7].

- **Denial of Service (DoS):** allows the user to make the system unavailable to real users through flooding it with traffic and requests [7].
- **Information Leak:** allows unauthorized individuals to access data that should be protected such as password, credit card data, personal information etc.

B. Probability x Impact Risk matrix

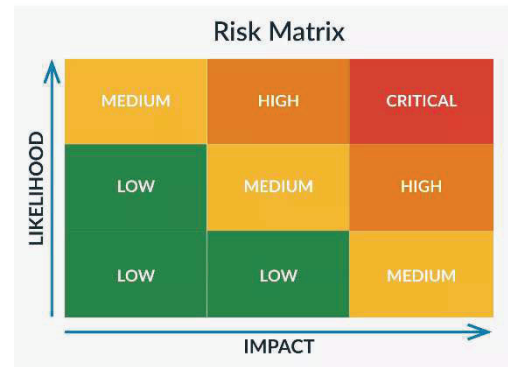


Fig. 2. Probability x Impact risk matrix.

As we can see on Fig. 2. the 'Probability x Impact' risk matrix is a tool used to assign risk levels based on the likelihood and impact of the vulnerability. This classification helps define the priorities for incident response as well as patching the vulnerabilities.

The likelihood criteria are defined by the weight of the steps required to exploit the vulnerability in the system. For example, if a vulnerability requires physical interaction with the target device, its probability is lower compared to a remotely exploitable vulnerability [8].

The impact represents the amount of damage that would happen if a certain vulnerability were exploited. The damage is a combination of the financial, legal, and reputational damages [8].

C. Common Vulnerability Scoring System (CVSS)

Created by the National Infrastructure Advisory Council in 2005, CVSS is a framework that gives a qualitative measure of severity of software vulnerability and their characteristics. It consists of 4 metric groups [9]:

- **Base:** represents the constant characteristics of the vulnerability
- **Threat:** refers to the set of characters that varies over time
- **Environmental:** indicated the characteristics that are user specific and are correlated to his environment
- **Supplemental:** give additional information but do not have an impact on the score

This framework gives a score from 0.0 to 10.0 to each vulnerability and then splits them based on a Qualitative severity rating scale. In Table II, we see a comparison of the scales for the versions v2.0, v3.x, and v4.0.

TABLE II. SEVERITY RATING SCALE FOR EACH CVSS VERSION [10]

	v2.0	v3.x	v4.0
0.0	Low	None	None
0.1 - 3.9		Low	Low
4.0 - 6.9	Medium	Medium	Medium
7.0 - 8.9	High	High	High
9.0 - 10.0		Critical	Critical

As we notice, during since v3.x, there was an introduction of 2 new levels on the severity scale, which are “None” and “Critical”, yet even if the CVSS specification accepts a score 0, NVD (National Vulnerability Database) doesn’t qualify elements with CVSS None as vulnerability since they have no impact [10].

In Fig. 3, we see the distribution by CVSS score of all Android CVEs discovered since its creation.

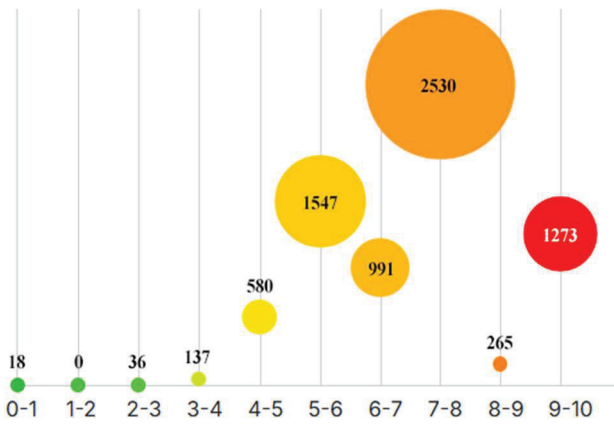


Fig. 3. Distribution by CVSS score of all Android CVEs.

When we map the 7377 [6] CVEs discovered in Android to CVSS v4.0 severity scale, we end up with an average of 7.3 falling in the High portion with a distribution as follows:

- **Low:** 191
- **Medium:** 3145
- **High:** 2795
- **Critical:** 1273

IV. EVOLUTION OF SECURITY MEASURES

A. The early versions (1.0 - 4.0)

The early versions of Android (from 1.0 to 4.4 [KitKat]) put the base for an operating system adapted to a broad number of devices and use cases. Yet, the technology and standards of that period restricted the security aspects.

Static permissions model: Permissions were defined on the application installation with limited flexibility for the user to update them afterward.

Lack of native encryption: Local data were mostly stored unencrypted, which made the users more at risk of physical attacks.

Common vulnerabilities: Most common attacks such as reverse engineering using tools like APKTool and malicious injections exploiting design flaws.

B. Progressive improvement (4.1 - 7.0)

From version 5.0 to 7.0, multiple improvements were brought to face the increase of the intensity and power of risks.

Introducing SELinux: SELinux (Security-Enhanced Linux) is a Linux Kernel security module that provides mechanisms for supporting access control security policies (including mandatory access control MAC). Android uses it to enforce MAC over all processes and improve the sandboxing of applications, limiting their actions on the system. SELinux implementation in Android evolved significantly since then, starting with Android 4.3 in Permissive mode, where it logged but did not enforce access denials, aiding in policy development. By Android 4.4, critical domains like install, netd, vold, and zygote shifted to partial Enforcing mode, enhancing security by actively restricting unauthorized access. From Android 5.0 onwards, SELinux operated in full Enforcing mode, covering over 60 domains, with init as the only process in the init domain, significantly reducing the attack surface. This evolution improved protection against privilege escalation attacks, as seen in cases where SELinux prevented malicious apps from accessing system resources beyond their sandbox, thereby mitigating risks like unauthorized data access and system compromise [11].

Runtime permissions (Android 6.0): Since Android 6.0, users have been able to manage permissions enabled for installed applications dynamically. This prevented the user from dealing with pre-installed apps from OEMs/carriers having non-changeable pre-granted permissions [12].

Android Keystore System: Android Keystore System was first introduced in Android 4.0 through the Keychain API, then through the keystore provider starting Android 4.3. It is meant for secure storage of encryption keys in specific containers which reduces the risk of sensitive data theft [13].

C. Secure ecosystem (8.0 - 12)

With the versions from 8.0 to 12, Google focused on improving the security of the whole Android ecosystem (system checks, play store, etc.).

Google play protect: This service scans the applications to identify malware. It is provided through two delivery methods. First is cloud-based protection, and its job is to scan applications rigorously before publication in the Google Play Store. Then comes on-device protections that scan the device daily for hints of potentially harmful applications.

Increased restrictions on sensitive APIs: Android restricted access to critical applications such as calls and SMS to reduce the abused exploitation.

SafetyNet API: This service verifies the integrity of devices and blocks access to compromised devices (such as rooted devices, fake users, etc.).

D. Latest versions (13 - 15)

The latest versions, starting Android 13, illustrated an increased attention to modern security needs and challenges.

One-time permissions: Users can give an application temporary permissions for services like the camera or location etc., which gives the user more control over his device.

Play Integrity API: Replacing SafetyNet, this API provides better detection for compromised devices and potential risks.

Advanced encryption: As a preventive way against the future risks related to encryptions with the introduction of quantum computers, Google started implementing PQC (post-quantum encryption) algorithms across all its products, including Android. Post-quantum cryptography (PQC) in Android is a critical advancement to counter future quantum computing threats, which could break traditional encryption methods like RSA and ECC using algorithms such as Shor's. Google is actively integrating PQC, with plans to adopt standardized algorithms like ML-KEM (formerly CRYSTALS-KYBER) for key exchange, as part of its broader transition to quantum-resistant security [14]. This implementation aims to protect sensitive communications and data storage on Android devices against "harvest now, decrypt later" attacks, where encrypted data is collected today for decryption once quantum computers are viable. The impact is significant, ensuring long-term security for Android users, especially in financial and personal data applications, with Google's Tink library providing experimental PQC support in C++ to facilitate developer adoption. An example is the potential use of ML-KEM in securing app-to-server communications, enhancing resistance to future quantum decryption capabilities [15].

E. The challenges

Even with the efforts of Google and every contributor in the Android Open-Source project, many challenges keep rising. Some of them come from undetected and patched vulnerabilities. Others are still out there due to many devices with end-of-support versions of Android [16].

Since the security support of Android 11 ended in Feb 2024, more than 900 million devices (around 32% [17] of the 3 billion users' pool [18]) are still in circulation with versions that are no longer getting security patches, which puts these users at a higher risk of attacks.

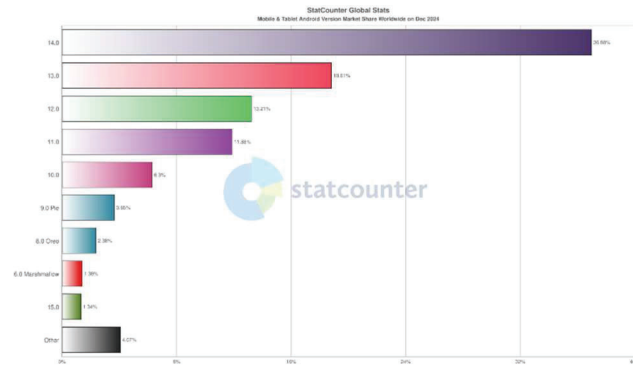


Fig. 4. Mobile & Tablet Android version market share Worldwide in Nov 2024.

Emerging threats pose new challenges to Android security as well, particularly with the rise of AI-driven attacks. These include AI-phishing campaigns that craft highly personalized emails or messages to deceive users into revealing sensitive information, such as login credentials or financial details, exploiting Android's open ecosystem. Another threat is AI-enhanced malware, which can adapt in real-time to evade detection, targeting Android devices through app stores or side-loaded applications[19].

Additionally, new vulnerable application that come from practices such as vibe coding have been spreading amongst users, which can put them at greater risk.

Proactive solutions are essential to address these challenges. Android is leveraging AI-based security features, such as Google Play Protect's Live Threat Detection, which uses on-device AI to monitor app behavior and flag suspicious activities, particularly those impersonating financial apps. This enhances real-time protection against AI-driven phishing and malware. Developers are encouraged to adopt secure coding practices and use AI-powered tools like Digital.ai Application Protection for Android, which offers runtime self-protection and threat alerting. Looking forward, Android may integrate advanced AI-driven anomaly detection to identify unusual device behavior and expand PQC implementations to safeguard against future quantum threats, ensuring a robust security framework. Collaboration with organizations like OWASP can further drive the development of standards to mitigate emerging risks, fostering a secure mobile ecosystem for years to come.

V. COMPARISON OF SECURITY FEATURES BETWEEN ANDROID, FLUTTER, AND REACT NATIVE

Modern frameworks of Android development have improved to respond to developer needs. With the ease of development and fast delivery provided abstraction etc., compromises are made in terms of performance, security, etc.

In Table 1, we compare Native Android, Flutter, and React Native in their support of security features.

TABLE III. COMPARISON BETWEEN NATIVE ANDROID, FLUTTER, AND REACT NATIVE OF THEIR SUPPORT OF MULTIPLE SECURITY FEATURES [20] [21] [3]

Feature	Android SDK	Flutter	React Native
Secure Storage	YES	YES	YES
Encryption API	YES	YES (lacks advanced features)	Need third-party libraries (Native code needed for complex cases)
Jailbroken/ Rooted Detection	YES	Need third-party libraries (less flexible than native)	Need third-party libraries (less accurate and flexible)
Local Authentication	YES	Need third-party libraries (Native for complex use cases)	Need third-party libraries (Native for complex use cases)
Connection Security	YES	YES	Need third-party libraries (Native code might be necessary)
Background Snapshot Prevention	YES	Need third-party libraries (require Native API calls)	NO (require modification on Native module)
Code Obfuscation	YES	YES (require native setup)	YES (Native + third party)
Secure Payment	YES	Need third-party libraries (Native code needed for complex cases)	Need third-party libraries (Native code needed for complex cases)

As we notice, cross-platform frameworks, most of the time, support the feature but for basic uses only and require third-party libraries or manual intervention in native code, etc. This compensates for the lack of security, but it brings with it new attack landscapes through vulnerabilities such as Dependency injection and misconfigurations.

VI. CONCLUSION AND FUTURE WORK

Android has come a long way in terms of security, from a basic OS into a robust and dynamic ecosystem. The improvements brought, such as Play Protect, Play Integrity API, etc., have drastically reduced the risk of attacks. However, many challenges are still faced, such as a lack of maturity in terms of security for multiplatform frameworks, the new risks that keep rising, and developers' knowledge about best practices which makes security an everlasting goal [22].

To move forward, a constant collaboration to improve the existing work provided by organizations like OWASP, ADA, etc. in terms of standards, and best practices and bring to light the base to improve built-in security features in frameworks, etc. Raising awareness for end users through warnings and restrictions can help prevent them from falling victim to cyber-attacks.

With the deprecation of JetSec-crypto library, future work will focus on researching and implementing a library to replace Secure Shared Preferences, following the recommendations of Google and legal requirements.

As field professionals, our goal of security isn't just to have a secure app, but to protect our users, through technical and human actions, we can create a relatively safe ecosystem for all of us [22].

REFERENCES

- [1] Ahmed Sherif, "Market share of mobile operating systems worldwide from 2009 to 2024, by quarter," Statista. Accessed: Jan. 02, 2025. [Online]. Available: <https://www.statista.com/statistics/272698/global-market-share-held-by-mobile-operating-systems-since-2009/>
- [2] A. Muñoz, "Cracking the Core: Hardware Vulnerabilities in Android Devices Unveiled," *Electronics (Switzerland)*, vol. 13, no. 21, Nov. 2024, doi: 10.3390/electronics13214269.
- [3] OWASP Foundation, "OWASP Mobile Application Security," OWASP MAS. Accessed: Jan. 03, 2025. [Online]. Available: <https://mas.owasp.org/MASVS/>
- [4] A. F. Abdul Kadir, A. Habibi Lashkari, and M. Daghmehchi Firoozjaei, "Android Operating System," 2024. doi: 10.1007/978-3-031-48865-8_2.
- [5] Y. Chen, "Research on Android Architecture and Application Development," in *Journal of Physics: Conference Series*, IOP Publishing Ltd, Aug. 2021. doi: 10.1088/1742-6596/1992/2/022168.
- [6] MITRE, "Google » Android (Operating system): Product details, threats and statistics," CVE Details. Accessed: Jan. 08, 2025. [Online]. Available: https://www.cvedetails.com/product/19997/Google-Android.html?vendor_id=1224
- [7] Z. Muhammad, Z. Anwar, A. R. Javed, B. Saleem, S. Abbas, and T. R. Gadekallu, "Smartphone Security and Privacy: A Survey on APTs, Sensor-Based Attacks, Side-Channel Attacks, Google Play Attacks, and Defenses," Jun. 01, 2023, *MDPI*. doi:10.3390/technologies11030076.
- [8] T. Murphy, "How to Evaluate Probability in Mobile App Vulnerabilities," NowSecure.
- [9] INC. FIRST, "Common Vulnerability Scoring System version 4.0 Specification Document," Forum of Incident Response and Security Teams official website. Accessed: Jan. 13, 2025. [Online]. Available: <https://www.first.org/cvss/specification-document>
- [10] NIST, "NVD - Vulnerability Metrics," Nist.gov. Accessed: Jan. 13, 2025. [Online]. Available: <https://nvd.nist.gov/vuln-metrics/cvss>
- [11] S. Smalley and R. Craig, "Security Enhanced (SE) Android: Bringing Flexible MAC to Android," in *20th Annual Network and Distributed System Security Symposium, NDSS 2013*, 2013.
- [12] "Runtime permissions," Android Documentation. Accessed: Jan. 07, 2025. [Online]. Available: https://source.android.com/docs/core/permissions/runtime_perms
- [13] T. Cooijmans, J. D. De Ruiter, and E. Poll, "Analysis of secure key storage solutions on android," in *Proceedings of the ACM Conference on Computer and Communications Security*, 2014. doi: 10.1145/2666620.2666627.
- [14] R. Hansen and P. Venables, "Post-Quantum Cryptography: Standards and Progress," Google Security blog. Accessed: Jan. 02, 2025. [Online]. Available: <https://security.googleblog.com/2024/08/post-quantum-cryptography-standards.html>
- [15] V. Lyskoit, "Post-quantum cryptography: The future of encryption," Nord VPN blog. Accessed: Apr. 02, 2025. [Online]. Available: <https://nordvpn.com/blog/post-quantum-cryptography/>
- [16] I. Sadiq, "Mobile Security Becomes Critical as Android Users Continue Using Handsets After Expiry," KBI Media. Accessed: Jan. 02, 2025. [Online]. Available: <https://kbi.media/mobile-security-becomes-critical-as-android-users-continue-using-handsets-after-expiry/>
- [17] statcounter GlobalStats, "Mobile Operating System Market Share Worldwide," Statcounter GlobalStats.
- [18] David Curry, "Android Statistics (2024)," 2024.
- [19] FBI San Francisco Media Office, "FBI Warns of Increasing Threat of Cyber Criminals Utilizing Artificial Intelligence," FBI.gov contact center. Accessed: Apr. 01, 2025. [Online]. Available: <https://www.fbi.gov/contact-us/field-offices/sanfrancisco/news/fbi-warns-of-increasing-threat-of-cyber-criminals-utilizing-artificial-intelligence>
- [20] R. Wu and T. Nguyen, "Mobile App Security: Native vs. Flutter," LinkedIn.
- [21] OWASP Foundation, "OWASP Mobile Security Project - Top 10 Mobile Risks," OWASP Official Documentation. Accessed: Dec. 30, 2024. [Online]. Available: <https://owasp.org/www-project-mobile-top-10/>
- [22] A. C. Cinar and T. B. Kara, "The current state and future of mobile security in the light of the recent mobile security threat reports," *Multimed Tools Appl*, vol. 82, no. 13, 2023, doi: 10.1007/s11042-023-14400-6.